Trillium Crash Course: Using a SciNet Cluster End to End

Companion to: `scinet-uoft-scientific-applications-ai-analyst.md` (§ "Read the SciNet docs for Trillium…") and `scinet-uoft-technical-concepts.md` §1 **Prepared:** 2026-10-16 · **Reading time:** about 15–20 minutes **Goal:** by the end you can talk through, fluently and correctly, *"log in, `module load`, build a venv on the login node, submit with `sbatch`, stage data to node-local disk, checkpoint because jobs have wall-time limits,"* and explain **why** each step exists.

Confidence markers. Facts without a marker come from Alliance/SciNet documentation or SciNet's Oct 2025 "Python on Trillium and Open OnDemand" handout (Ramses van Zon). *(verify)* means it's standard Alliance practice or Niagara-era behaviour that I couldn't confirm for Trillium specifically. Run the command or check docs.scinet.utoronto.ca before stating it as fact.

An honesty note. Your profile has Docker and multi-GPU training, but no Slurm time. Don't say "on Trillium I did X." Say **"the workflow is…"** or **"what I'd tell a user is…"** and lean on the anchors in §10. Explaining the *why* well reads as more senior than claiming hours anyway.

---

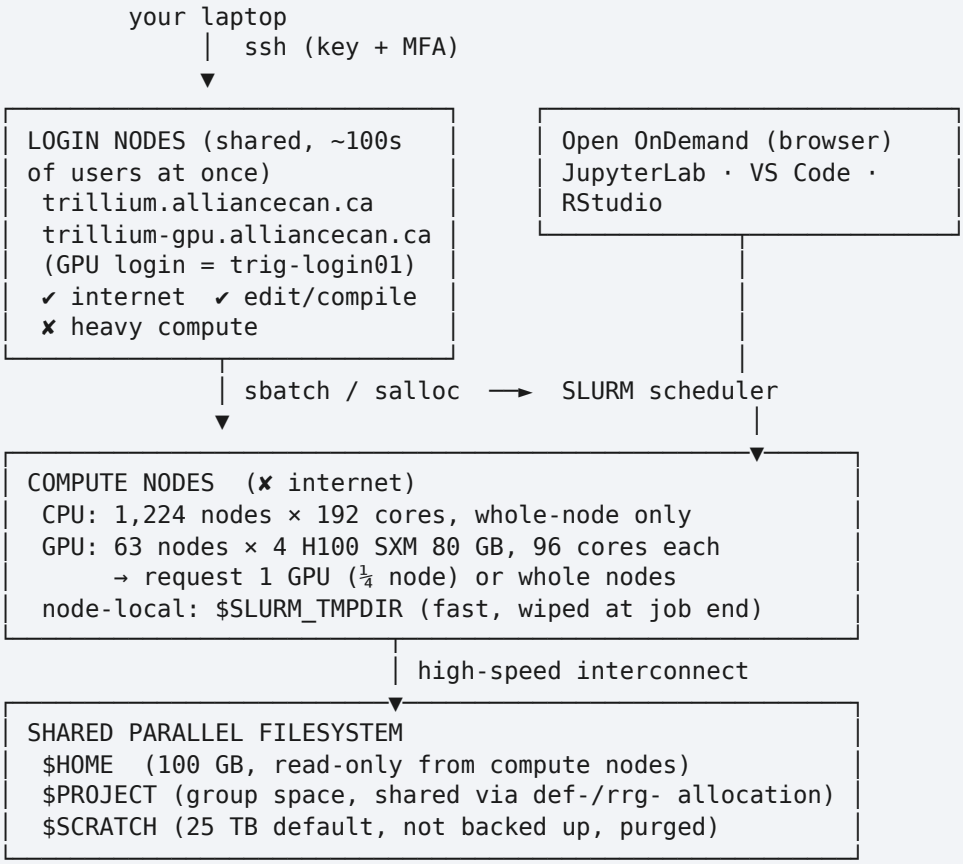
0. The 60-second version (memorize this paragraph) · *1 min*

> "You SSH into a **login node**. It's shared, it has internet, and it's for editing, compiling, installing and submitting. It's never for heavy compute. You set up software with **Lmod modules**, e.g. `module load python/3.13`, and build a **virtualenv** on the login node from the Alliance's pre-built **wheels** with `pip install --no-index`, rather than conda. Code and the venv live in **home** or **project**. Big data and job output go to **scratch**, which is large and fast but not backed up and gets purged. You write a **Slurm batch script** that asks for nodes or GPUs and a wall time, and submit it with `sbatch`. The job runs on **compute nodes**, which have **no internet**, so everything must be downloaded beforehand. Inside the job you **stage** the dataset onto **node-local storage** (`$SLURM_TMPDIR`) so thousands of small reads don't hammer the shared filesystem. Because Trillium caps jobs at **24 hours**, long training **checkpoints** regularly and **resumes**, and you chain jobs with `--dependency`. For interactive work there's **Open OnDemand** (JupyterLab, VS Code). If the software stack is awkward, you use an **Apptainer** container, not Docker."

The rest of this document unpacks each sentence.

---

1. The mental model: what's physically there · *3 min*



The three ideas behind every rule:

1. **Login nodes are shared.** If one user runs a 40-minute `python train.py` there, every other user's `ls` and `vim` slows down. SciNet kills such processes. This is the #1 new-user ticket.
2. **Compute nodes are isolated.** No internet (Trillium docs say so explicitly), so `pip install`, `huggingface-cli download` and `wget` inside a job fail. Everything is fetched on the login node first.
3. **The shared filesystem is shared too.** It's fast for big sequential reads and writes, but slow for millions of tiny files opened by many jobs at once (metadata load). Hence staging to node-local disk and packing datasets into a few big files.

Numbers to have ready: 241,056 CPU cores + 252 H100 GPUs. Launched Aug 2025, replacing Niagara (CPU) and Mist (GPU). Jobs run **15 min to 24 h**.

---

2. Logging in and first-time setup · *1.5 min*

1. You need an **Alliance CCDB account** (sponsored by a PI). Then request **Trillium access** on the CCDB "Access Systems" page. It takes about an hour to provision.
2. Register an **SSH key** in CCDB and set up **multifactor authentication** (Alliance-wide requirement) *(verify)*.
3. Log in:

```
ssh -i ~/.ssh/id_ed25519 USERNAME@trillium.alliancecan.ca       # CPU side
ssh -i ~/.ssh/id_ed25519 USERNAME@trillium-gpu.alliancecan.ca   # GPU side (trig-login01)
```

4. First time only, run `trisetup`. It sets up your shell startup files and creates symlinks to your scratch and project directories.

Why two login nodes? The GPU subcluster has its own login node, so you build and test GPU software (CUDA builds, PyTorch import checks) on a machine with the same environment as the GPU compute nodes. *Say it:* "Build where you'll run: GPU work from the GPU login node."

---

3. Filesystems: where things go · *3 min*

	`$HOME`	`$PROJECT`	`$SCRATCH`	`$SLURM_TMPDIR` (node-local)
Size	100 GB default	per group allocation	25 TB default	per node, job lifetime only
Backed up	yes *(verify)*	yes/persistent *(verify)*	**no**	no, wiped at job end
Purged	no	no	**yes**, files untouched for a while (Niagara was ~2 months) *(verify Trillium window)*	always, at job end
From compute nodes	**read-only**	read/write	read/write	read/write, fastest
Put here	dotfiles, small scripts, venvs *(see note)*	shared code, group datasets, final results, venvs	datasets in use, checkpoints, job output, logs	the copy of the dataset a job is reading *right now*

The four rules you'd tell a user:

- **Jobs write to scratch (or project), never home.** Home is read-only on Trillium compute nodes, so a job that writes `./output` from a home directory fails. That's a very common ticket.
- **Scratch is not storage.** Not backed up and purged. Copy anything you'd cry about to project, or off-site (Globus).
- **Quotas count files as well as bytes** *(standard Alliance; check with `diskusage_report` or SciNet's quota tool (verify the name))*. A conda install or an unpacked dataset with 2 million JPEGs can blow the file-count quota while using very little space.
- **Many small files is the enemy.** Pack them: `tar`, HDF5, Zarr, WebDataset shards, or a single `.npy` memmap. Then stage the packed file to node-local disk and unpack it there.

Node-local caveat to know: on Niagara, compute nodes had **no local disk**, so `$SLURM_TMPDIR` was a **RAM disk** that ate into job memory. On other Alliance clusters it's a local SSD. Whether Trillium's is RAM- or disk-backed is *(verify)*. It's a great question to ask in the interview: *"Is `$SLURM_TMPDIR` on Trillium a RAM disk like Niagara, or local NVMe? That changes what I'd tell users about staging large datasets."*

Big transfers: use **Globus** (resumable, checksummed, runs in the background) rather than `scp` for anything over a few GB.

---

4. Software: Lmod modules · *2 min*

The cluster doesn't `apt install` things for you. Software is installed centrally, in many versions side by side, and you **load** what you need into your environment (it edits `PATH`, `LD_LIBRARY_PATH` and so on).

```
module avail                    # what can be loaded right now
module spider python            # search everything, all versions
module spider python/3.13       # what must be loaded first to get this version
module load python/3.13         # load it
module load StdEnv/2023 gcc cuda   # Alliance-style stack (on Trillium: verify exact names)
module list                     # what's loaded
module purge                    # unload everything; start clean
module save mystack; module restore mystack   # named collections
```

Concepts worth one sentence each:

- **Lmod hierarchy:** some modules appear only after you load their compiler or MPI (e.g. `openmpi` appears after `gcc`). That's why `module avail` doesn't show everything and `module spider` is the real search.
- **StdEnv** = a versioned snapshot of the whole Alliance software stack (compilers, libraries, Python wheels). `StdEnv/2023` is current. Pinning it keeps a project reproducible for years.
- **CVMFS:** the Alliance stack is distributed read-only to every cluster over CVMFS, so the same `module load` line works on Fir, Nibi, Rorqual, Narval and Trillium.
- **Trillium's stack** *(verify)*: on Niagara you had to opt in to the Alliance stack with `module load CCEnv StdEnv`. SciNet's Teach docs say CCEnv isn't needed there, and Trillium is an Alliance national system. Check `module list` right after login to see what's default.
- **H100 gotcha:** old PyTorch/CUDA builds don't support H100 (compute capability 9.0). Users porting 2021-era environments hit this. Newer wheels fix it.

Say it: "Modules give you centrally built, CPU-optimized software in pinned versions. I'd put the exact `module load` line at the top of every job script so the job doesn't depend on whatever the user had loaded in their shell."

---

5. Python on the cluster: venv + wheels, not conda · *3 min*

The recipe (do this on the login node):

```
module load python/3.13                     # or: module load StdEnv/2023 python/3.11
virtualenv --no-download $HOME/venvs/torch   # SciNet often suggests $PROJECT or $SCRATCH for bigger envs (verify)
source $HOME/venvs/torch/bin/activate
pip install --no-index --upgrade pip
pip install --no-index torch torchvision numpy pandas   # from the Alliance wheelhouse
avail_wheels torch                          # which builds of a package are available
pip freeze > requirements.txt               # record it
```

What `--no-index` means: "don't go to PyPI; install only from the local wheelhouse." The Alliance's wheels are **pre-built for the cluster's CPUs and CUDA**, compiled against the module stack's libraries. They install in seconds and are optimized. If a package isn't in the wheelhouse, drop `--no-index` for that one package on the login node (it has internet), or ask support to add a wheel.

Why conda is discouraged (the Alliance's own reasons, a likely interview question):

1. **It duplicates the module stack:** a second copy of compilers, MPI, CUDA and BLAS, which then conflicts with the system ones.
2. **Not optimized:** generic binaries, not built for the cluster's AVX-512 / Zen CPUs.
3. **File-count quota:** a conda install is hundreds of thousands of small files in `$HOME`, which eats the file quota and stresses the shared filesystem.
4. **Slow:** dependency solving and installing are much slower than wheels.
5. **It edits `~/.bashrc`** (`conda init`), which silently changes the environment of every later job and breaks modules.

The Alliance's escape hatch for "I really need conda" is **put it in an Apptainer container** (§6).

Inside the job: load the *same* modules, then `source …/bin/activate`. For many-node jobs, a well-known Alliance pattern is to **build the venv in `$SLURM_TMPDIR` at job start** from the wheelhouse (fast, no shared-FS metadata storm) *(standard Alliance pattern)*.

Ticket you'll get: *"pip install works on the login node but my job says `ModuleNotFoundError`."* → The job didn't load the same `python` module or didn't activate the venv; or the venv was built with a different Python version than the job loads.

---

6. Apptainer: containers on HPC · *2 min*

Why not Docker? Docker needs a **root-owned daemon**. On a shared cluster, giving users access to it effectively gives them root. **Apptainer** (formerly Singularity) runs containers **as you, rootless**, with no daemon. The image is a **single `.sif` file**: one file on the parallel filesystem instead of thousands, so it's fast and quota-friendly.

```
module load apptainer
# Convert a Docker/NGC image (on the login node: needs internet)
apptainer build pytorch.sif docker://nvcr.io/nvidia/pytorch:24.08-py3
# Run on a GPU compute node
apptainer exec --nv \
  --bind $SCRATCH:/scratch,$SLURM_TMPDIR:/data \
  pytorch.sif python train.py --data /data
```

- `--nv` passes the host NVIDIA driver and GPUs into the container.
- `--bind` mounts host directories (scratch, project, node-local) into the container. Home and the current directory are usually bound by default.
- **When to use it:** conda-only stacks, complex or ancient dependency chains, "it works in this Docker image from the paper," and reproducibility (archive the `.sif` with the paper).
- **When not to:** if the wheelhouse has everything, a venv is simpler and gets the cluster-optimized builds.

Your anchor: you've Dockerized pipelines (melodiq: docker-compose, Airflow). *"I've packaged pipelines with Docker. On HPC the same image converts straight to an Apptainer SIF and runs rootless, which is exactly the path I'd show a user who arrives with a Dockerfile."*

---

7. Submitting jobs with Slurm · *3 min*

Scheduling rules on Trillium:

- **CPU side: whole nodes only.** You get all 192 cores and all the memory, so you don't specify `--mem`. Small serial work should be **bundled** (GNU Parallel, job arrays packing tasks) to fill a node.
- **GPU side: one GPU (¼ node, 24 cores) or whole nodes (4 GPUs).** Single-GPU jobs are the common case for users.
- **Wall time: 15 min to 24 h.** A **debug** partition (`-p debug`) gives quick turnaround for short tests.
- **Accounts:** `--account=def-yourpi` (default allocation) or `rrg-…` (Resource Allocation Competition award). Fair-share sets priority.

An annotated single-GPU training script:

```
#!/bin/bash
#SBATCH --job-name=train-resnet
#SBATCH --account=def-mypi
#SBATCH --gpus-per-node=1          # one H100 = 1/4 node; =4 for a whole node
#SBATCH --time=12:00:00            # ≤ 24h; shorter requests backfill sooner
#SBATCH --output=%x-%j.out         # job name + job ID
#SBATCH --signal=B:USR1@300        # warn the script 5 min before the wall-time limit

module purge
module load python/3.13            # same modules the venv was built with
source $PROJECT/venvs/torch/bin/activate

# 1) stage data: one big sequential read from scratch → node-local
tar -xf $SCRATCH/datasets/imagenet-subset.tar -C $SLURM_TMPDIR

# 2) run; checkpoints go to scratch (survive the job), data comes from node-local
cd $SCRATCH/runs/resnet
srun python train.py \
     --data $SLURM_TMPDIR/imagenet-subset \
     --ckpt-dir $SCRATCH/runs/resnet/ckpt \
     --resume auto

# 3) copy anything written only to node-local back before the job ends
```

Notes: whole-node GPU jobs use `--gpus-per-node=4` plus `torchrun` or `srun` with one task per GPU. Some sources show `--gres=gpu:1`; both are Slurm syntax, and the SciNet handout uses `--gpus-per-node`. Multi-node: `--nodes=2 --gpus-per-node=4`, with NCCL over InfiniBand.

The everyday command set:

Command	Use
`sbatch job.sh`	submit; prints the job ID
`squeue -u $USER` / `squeue --start -j ID`	where's my job / when will it start
`scontrol show job ID`	full detail, incl. **Reason** it's pending (`Priority`, `Resources`, `QOSMaxWallDurationPerJobLimit`…)
`scancel ID`	cancel
`sacct -j ID --format=JobID,State,Elapsed,MaxRSS`	what happened after it ended
`seff ID`	efficiency: did it use what it asked for?
`salloc --gpus-per-node=1 --time=1:00:00` (or `-p debug`)	interactive session on a compute node for debugging
`sbatch --array=0-19%5 job.sh`	20 configs, ≤5 at once; `$SLURM_ARRAY_TASK_ID` picks one
`sbatch --dependency=afterok:ID next.sh`	run only after job ID succeeds (pipelines, chained training)

Monitoring a GPU job: while it runs, `srun --jobid=ID --overlap --pty nvidia-smi` (or SSH to the node, if SciNet allows it *(verify)*) shows GPU utilization. **Low GPU utilization usually means the data loader is starving the GPU**: too few `num_workers`, reading small files from the shared filesystem, or CPU-side augmentation. Staging and more workers fix most of it.

---

8. Staging data and checkpointing · *2 min*

Staging (why and how)

- **Why:** 1,000 jobs each opening 1M small files on the shared filesystem means a metadata storm that slows the *whole cluster*. One sequential read of a tarball per job, then local reads, is fast for you and kind to everyone.
- **How:** keep the dataset packed in scratch (`.tar`, HDF5, Zarr, WebDataset shards). At job start: `tar -xf $SCRATCH/data.tar -C $SLURM_TMPDIR`. Point the data loader at `$SLURM_TMPDIR`. At the end, copy results back to scratch or project. Node-local is wiped when the job ends.
- **RAM-disk caveat:** if `$SLURM_TMPDIR` is RAM-backed *(Niagara was; verify Trillium)*, the staged data counts against node memory. For huge datasets, read packed shards straight from scratch instead.

Checkpointing (why and how)

- **Why:** the 24-hour wall-time limit kills any job that runs longer. Nodes also fail. Without a checkpoint, a 3-day training run can't finish at all.
- **What to save:** model weights **plus** optimizer state, LR-scheduler state, epoch/step, RNG states and the data-loader position. Weights alone make the resumed run diverge from the original.
- **How often:** every N minutes or steps, cheap enough that losing one interval doesn't matter. Write to a temp file and then `rename` it, so a kill mid-write never corrupts the last good checkpoint.
- **Chaining:** submit job 2 with `--dependency=afterany:<job1>` (afterany, because hitting the time limit counts as a failure). Or catch the `--signal=B:USR1@300` warning, save, and `sbatch` the next segment from inside the script.

```python
# minimal resume pattern
ckpt = Path(args.ckpt_dir) / "last.pt"
if ckpt.exists():
    s = torch.load(ckpt, map_location="cuda")
    model.load_state_dict(s["model"]); opt.load_state_dict(s["opt"])
    sched.load_state_dict(s["sched"]); start_step = s["step"] + 1
for step in range(start_step, total_steps):
    ...
    if step % 1000 == 0:
        tmp = ckpt.with_suffix(".tmp")
        torch.save({"model": model.state_dict(), "opt": opt.state_dict(),
                    "sched": sched.state_dict(), "step": step}, tmp)
        tmp.rename(ckpt)                        # atomic replace
```

Your anchor: VarianceEngine. You fine-tuned MusicGen-medium with LoRA across two GPUs and have a checkpoint at step 1600 that training resumes from. *"I've had to resume long fine-tuning runs from checkpoints with optimizer state, so building checkpoint/restart around a 24-hour wall time is the same habit, enforced by the scheduler."*

---

9. Interactive work: Open OnDemand / Jupyter · *1 min*

- SciNet's **Open OnDemand** portal (`ondemand.scinet.utoronto.ca`) gives browser-based **JupyterLab, VS Code, RStudio** and a file manager, running as a scheduled job on Trillium. This is SciNet's equivalent of the JupyterHub that other Alliance clusters run.
- **To use your own venv as a Jupyter kernel:** activate it in a terminal and run **`venv2jup`** (SciNet helper). The venv then appears as a kernel.
- **What to tell users:** notebooks are for exploring, debugging and plotting. Anything over an hour, or any sweep, becomes an `sbatch` script. An idle notebook holding an H100 wastes a quarter of a node.

---

10. Saying it "as if you've done it" (honestly) · *2 min*

The walkthrough, first person and true:

> "The workflow I'd follow, and what I'd walk a new user through: SSH to the GPU login node and `module load` the Python version. Build a virtualenv there from the Alliance wheelhouse with `pip install --no-index`, and download models and datasets there too, because compute nodes have no internet. Code and the env go in project, data and outputs in scratch. Then I write the batch script: one H100, a realistic wall time, the same `module load` and `source activate` lines, untar the dataset into `$SLURM_TMPDIR`, train with checkpoints to scratch. Then `sbatch` it, check `squeue`, and afterwards `seff` to see if I over-asked. Because of the 24-hour cap, the training loop saves full state regularly and resumes automatically, and I chain the segments with dependencies. I've done the checkpoint-and-resume part on my own multi-GPU LoRA fine-tuning. On a cluster the scheduler just makes it mandatory."

If asked "Have you used Slurm?": *"Not in production. My GPU work has been on my own two-GPU machine. But I've worked through the Trillium and Alliance docs closely enough to explain the why*

behind each step (no internet on compute nodes, home read-only from jobs, wheels over conda, staging, checkpointing), and I'd validate all of it hands-on in the first weeks."

---

11. Quick-fire drill (cover the answers) · *2 min*

1. **User: "My job can't `pip install`."** → No internet on compute nodes. Install on the login node into a venv with `--no-index`, then activate it in the job.
2. **User: "My job dies writing output."** → Probably writing under `$HOME`, which is read-only from compute nodes. Write to `$SCRATCH`.
3. **"Why not conda?"** → Duplicates the module stack, unoptimized binaries, huge file counts against the quota, slow, edits `.bashrc`. Use venv + wheels, or conda inside Apptainer.
4. **"Why not Docker?"** → Needs a root daemon. Apptainer is rootless, single SIF file, `--nv` for GPUs, and converts Docker images.
5. **"My GPU sits at 20% utilization."** → Data-loader bound. Stage to `$SLURM_TMPDIR`, pack small files, raise `num_workers`, check with `nvidia-smi`/profiler.
6. **"My job waits forever."** → `scontrol show job` for the Reason. Ask for a shorter, accurate wall time (backfill), one GPU instead of four if the code doesn't scale, and check fair-share.
7. **"Training needs 3 days."** → 24-hour cap. Checkpoint full state, resume, chain with `--dependency=afterany`.
8. **"I need 500 tiny CPU runs."** → Trillium CPU jobs are whole-node (192 cores). Bundle them with GNU Parallel or a packed array, not 500 one-core jobs.
9. **"Where should my 2 TB dataset live?"** → Packed in scratch while in use (purged, so keep the master copy in project or off-site). Transfer with Globus.
10. **"What's `module spider` for?"** → Searching the full Lmod hierarchy, including versions and what must be loaded first.

---

12. Before the interview: verify these (10 minutes on docs.scinet.utoronto.ca)

- [ ] Trillium **scratch purge window** and whether home/project are backed up
- [ ] **`$SLURM_TMPDIR`**: RAM disk or local NVMe on Trillium GPU nodes, and how large
- [ ] **Default module stack** after login: is `StdEnv/2023` loaded, or is it SciNet's own stack + `CCEnv`?
- [ ] **Debug partition** limits (max time, GPUs) and the exact GPU flag in the quickstart example
- [ ] Quota-checking command on Trillium
- [ ] Whether SSH to a running job's compute node is allowed (for `nvidia-smi`)

Sources: Alliance Trillium Quickstart · SciNet Trillium Quickstart · Transition from Niagara to Trillium · Alliance Running jobs · Alliance Anaconda policy · Available Python wheels · SciNet "Python on Trillium and Open OnDemand" handout, Oct 2025 · SciNet: Installing your own Python modules (venv2jup) · Trillium Neptune nodes (no internet on compute) · SciNet Data Management (Niagara-era $SLURM_TMPDIR/purge) · Alliance service catalogue: Trillium · Mila DRAC cluster notes